

 Universidad Católica de Cuenca	LISTA DE COTEJO PARA LA EVALUACIÓN DEL PROYECTO RETO
---	---

**UNIDAD ACADÉMICA DE INFORMÁTICA, CIENCIAS DE LA
COMPUTACIÓN, E INNOVACIÓN TECNOLÓGICA**
CARRERA DE SISTEMAS BIOMÉDICOS
Período Lectivo MARZO 2026 – AGOSTO 2026

Asignatura:	Infraestructura de Sistemas Clínicos en la Nube – Gestión de Proyectos Tecnológicos Sanitarios	Fecha:	20 de julio del 2026
Docente:	Ing. Fanny Chuchuca, MSc. Ing. Pablo Salamea, Msc.		
Ciclo:	7mo	Bloque Temático N°:	3
TEMA:	Diseño e Implementación de un Repositorio Clínico Único en la Nube para una Red Hospitalaria Oncológica Multi-Sede		
Nombre del estudiante:			

AVANCE 1 — Diseño inicial y microservicios base
Repositorio Clínico Único en la Nube para una Red Hospitalaria Oncológica Multi-Sede

1. Objetivo del avance

El primer avance tuvo como objetivo plantear la arquitectura inicial del repositorio medico regional de SOLCA y construir los microservicios base de forma independiente. La necesidad principal era resolver la fragmentacion de informacion clinica entre SOLCA Cuenca, SOLCA Manabi y SOLCA Quito, donde cada sede puede tener datos separados de pacientes, consultas, laboratorio e imagenologia.

2. Analisis del problema actual

El problema identificado es que la informacion oncologica se encuentra distribuida entre sedes y dominios clinicos. Un paciente puede tener datos administrativos en una sede, consultas en otra, resultados de laboratorio en otra fuente y estudios de imagenologia en otro sistema. Esto dificulta la continuidad asistencial, genera duplicidad de pacientes y obliga al personal medico a revisar varias fuentes para reconstruir la historia regional.

Para responder a ese problema se diseno un repositorio regional basado en microservicios. Cada microservicio mantiene su propio dominio y su propia base de datos, mientras el Repositorio Clinico Regional funciona como punto de consulta central. En esta primera fase el repositorio podia existir sin integracion completa, dejando preparada la base para el Avance 2.

3. Que hicimos

Se crearon microservicios Spring Boot separados para los dominios solicitados:

Microservicio	Carpeta	Puerto	Base de datos
Paciente Maestro Regional	paciente-maestro-service	8081	paciente_maestro_db
Consulta Clinica	consulta-clinica-service	8082	consulta_clinica_db
Laboratorio Clinico	laboratorio-clinico-service	8083	laboratorio_clinico_db
Imagenologia / PACS basico	imagenologia-service	8084	imagenologia_db
Repositorio Clinico Regional	repositorio-clinico-regional-service	8085	repositorio_clinico_regional_db

Tambien se agrego una estructura de base de datos independiente mediante database/init/01-create-databases.sql, donde cada dominio clinico tiene una base PostgreSQL separada.

4. Como lo hicimos

Cada microservicio se implemento con:

- Java 17 y Spring Boot.
- API REST basica por dominio.
- Spring Data JPA para persistencia.
- PostgreSQL como motor de base de datos.
- application.properties independiente por servicio.
- Datos semilla en data.sql para probar rapidamente los endpoints.
- Dockerfile individual para permitir construccion aislada.

La organizacion del codigo permite ejecutar cada servicio de manera separada o levantar todo el entorno con docker-compose.yml.

5. Diseno preliminar de arquitectura cloud

La arquitectura preliminar se documento con diagramas generados en la carpeta documentacion/assets/:

- diagrama_arquitectura_cloud_solca.png
- diagrama_componentes_cloud_solca.png
- diagrama_modelo_datos_solca.png

El diseno considera una aplicacion frontend, microservicios backend, bases de datos independientes y un repositorio regional como agregador. En esta fase se definio la separacion de responsabilidades, aunque la integracion completa se trabajo en el Avance 2.

6. Modelo del Paciente Maestro Regional

El Paciente Maestro Regional se definio como la identidad minima comun para relacionar los registros clinicos entre sedes. El modelo se implemento en:

paciente-maestro-service/src/main/java/ec/edu/solca/paciente/model/Paciente.java

Campos principales:

- idPacienteRegional
- cedula
- nombres
- apellidos
- sedeOrigen
- fechaNacimiento
- sexo

- direccion
- telefono

Este identificador regional permite que consultas, laboratorio e imagenologia se vinculen al mismo paciente sin depender de una sola base centralizada.

7. Diseno de bases de datos independientes

La independencia de datos se cumple con bases separadas:

```
CREATE DATABASE paciente_maestro_db;
CREATE DATABASE consulta_clinica_db;
CREATE DATABASE laboratorio_clinico_db;
CREATE DATABASE imagenologia_db;
CREATE DATABASE repositorio_clinico_regional_db;
```

Cada microservicio se conecta solo a su base mediante su propia variable SPRING_DATASOURCE_URL. Esto cumple el principio de base de datos por microservicio.

8. Componentes cloud identificados

Componente	Uso planteado en el proyecto
IaaS	VPS o maquinas virtuales para ejecutar contenedores cuando se requiere control de infraestructura.
PaaS	Plataforma para desplegar microservicios Spring Boot y frontend sin administrar servidores manualmente.
SaaS	GitHub para repositorios, Postman para pruebas y herramientas de colaboracion.
DBaaS	PostgreSQL administrado para alojar las bases independientes de cada microservicio.

9. Endpoints minimos funcionales

Servicio	Endpoints principales
Paciente Maestro	GET /pacientes, GET /pacientes/{id}, GET /pacientes/cedula/{cedula}, POST /pacientes
Consulta Clinica	GET /consultas, GET /consultas/paciente/{idPacienteRegional}, POST /consultas
Laboratorio Clinico	GET /laboratorio, GET /laboratorio/paciente/{idPacienteRegional}, POST /laboratorio
Imagenologia	GET /imagenes, GET /imagenes/paciente/{idPacienteRegional}, POST /imagenes
Repositorio Regional	Servicio creado para preparar la integracion posterior

10. Evidencias disponibles

Evidencia solicitada	Donde esta en el proyecto
Repositorio GitHub creado	Backend y frontend publicados en GitHub.
Codigo organizado por microservicios	Carpetas paciente-maestro-service, consulta-clinica-service, laboratorio-clinico-service, imagenologia-service, repositorio-clinico-regional-service.
Pruebas Postman	postman/SOLCA-Repositorio-Clinico-Regional.postman_collection.json.
Capturas de bases de datos	Se pueden tomar desde PostgreSQL/psql usando las bases creadas en database/init/01-create-databases.sql.

Evidencia solicitada	Donde esta en el proyecto
Diagrama preliminar	documentacion/assets/diagrama_arquitectura_cloud_solca.png y documentacion/assets/diagrama_modelo_datos_solca.png.

11. Cumplimiento de la rubrica

Requisito	Estado	Explicacion
Análisis del problema SOLCA Cuenca, Manabí y Quito	Cumple	Se documentó la fragmentación de información entre sedes y la necesidad de identidad regional.
Diagrama preliminar de arquitectura cloud	Cumple	Existe diagrama de arquitectura cloud en documentacion/assets.
Modelo del Paciente Maestro Regional	Cumple	Implementado como entidad Paciente y documentado como identidad regional.
Bases de datos independientes	Cumple	Cada microservicio usa una base PostgreSQL propia.
Componentes IaaS, PaaS, SaaS, DBaaS	Cumple	Identificados en la documentación y diagramas cloud.
Microservicios creados y funcionando	Cumple	Existen cinco microservicios base con API REST y persistencia.
API REST básica	Cumple	Cada servicio expone endpoints GET/POST mínimos.
Pruebas en Postman	Cumple parcialmente	Existe colección Postman; las capturas deben tomarse al ejecutar las pruebas.
Capturas de bases de datos	Pendiente de evidencia visual	Las bases existen; faltan capturas guardadas como imagen.

12. Conclusion

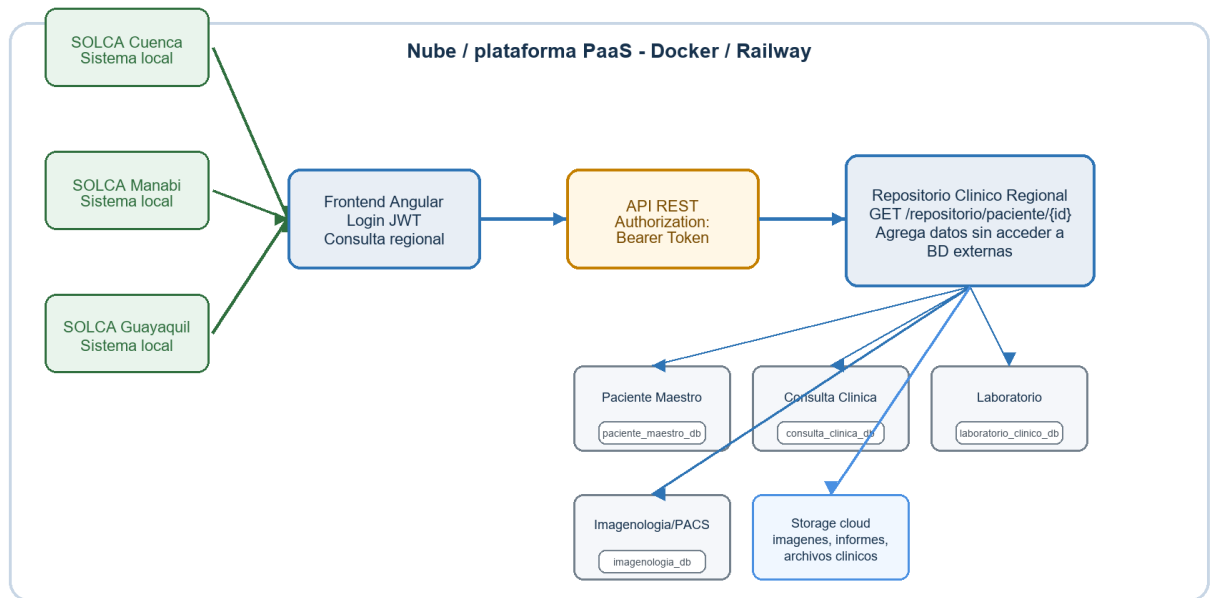
El Avance 1 queda cubierto porque el proyecto define la arquitectura inicial, separa los dominios clínicos en microservicios independientes, crea bases PostgreSQL separadas y deja disponible una colección Postman para pruebas. Las únicas evidencias que deben anexarse manualmente son capturas visuales de Postman y de las bases de datos durante la ejecución.

Anexos: diagramas

Como evidencia del diseño e implementación se incluyen los diagramas generados para el proyecto.

Diagrama preliminar de arquitectura cloud

Arquitectura cloud preliminar - Repositorio Clinico Regional SOLCA

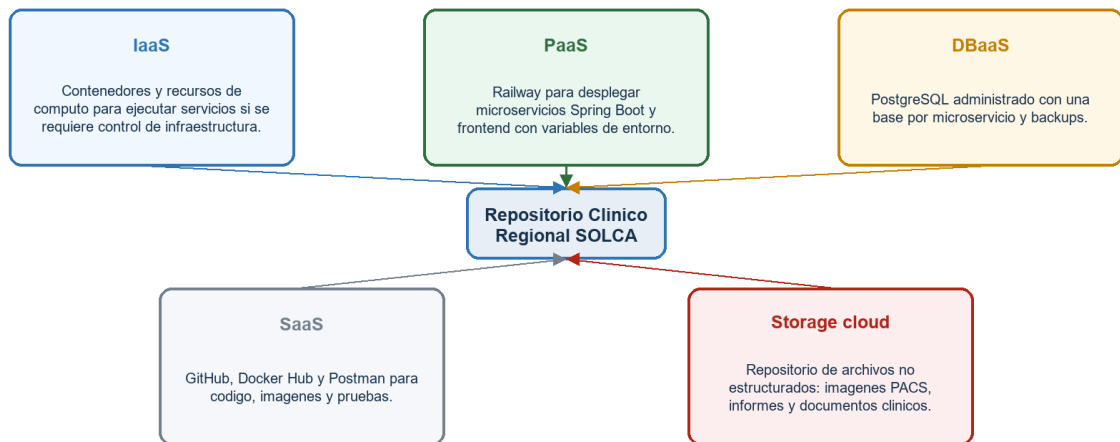


Principio clave: cada microservicio conserva su propia base de datos; la integración se hace por APIs REST.

Diagrama preliminar de arquitectura cloud

Componentes cloud identificados

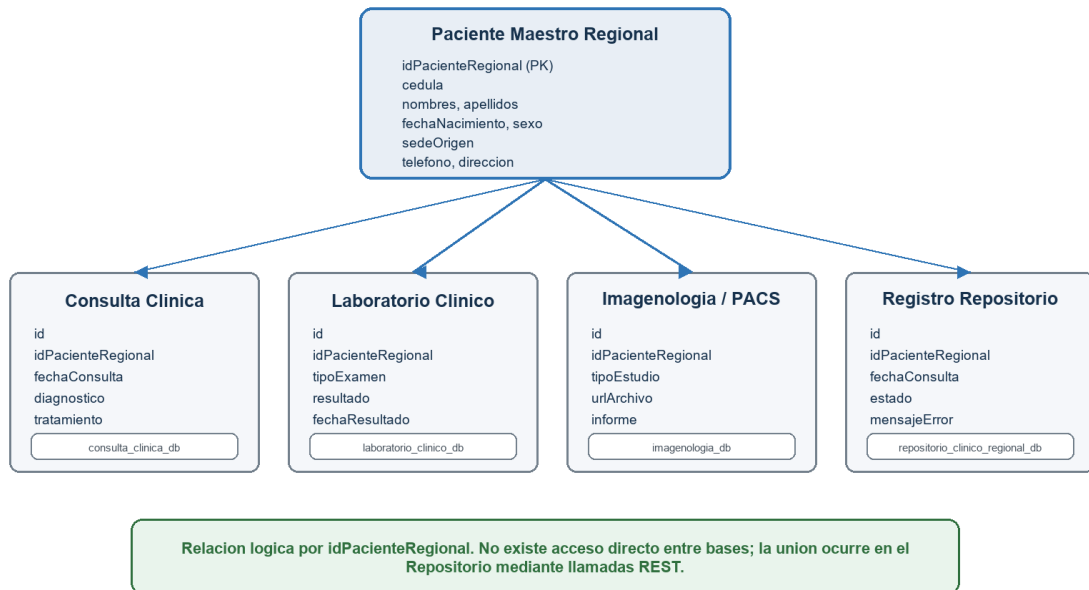
Componentes cloud identificados para el proyecto



Componentes cloud identificados

Modelo de datos del Paciente Maestro Regional

Modelo de datos: Paciente Maestro y bases independientes



Modelo de datos del Paciente Maestro Regional

 Universidad Católica de Cuenca	LISTA DE COTEJO PARA LA EVALUACIÓN DEL PROYECTO RETO
---	---

**UNIDAD ACADÉMICA DE INFORMÁTICA, CIENCIAS DE LA
COMPUTACIÓN, E INNOVACIÓN TECNOLÓGICA**
CARRERA DE SISTEMAS BIOMÉDICOS
Período Lectivo MARZO 2026 – AGOSTO 2026

Asignatura:	Infraestructura de Sistemas Clínicos en la Nube – Gestión de Proyectos Tecnológicos Sanitarios	Fecha:	20 de julio del 2026
Docente:	Ing. Fanny Chuchuca, MSc. Ing. Pablo Salamea, Msc.		
Ciclo:	7mo	Bloque Temático N°:	3
TEMA:	Diseño e Implementación de un Repositorio Clínico Único en la Nube para una Red Hospitalaria Oncológica Multi-Sede		
Nombre del estudiante:			

AVANCE 2 — Repositorio Clínico Regional e integración
Repositorio Clínico Único en la Nube para una Red Hospitalaria Oncológica
Multi-Sede

1. Objetivo del avance

El segundo avance tuvo como objetivo convertir el Repositorio Clínico Regional en un agregador real. En esta fase el repositorio ya no solo existe como microservicio base, sino que consume los microservicios de Paciente Maestro, Consulta Clínica, Laboratorio e Imagenología mediante APIs REST y devuelve una respuesta consolidada por paciente.

2. Que hicimos

Se implemento la integracion del microservicio repositorio-clinico-regional-service con los microservicios clinicos:

- paciente-maestro-service
- consulta-clinica-service
- laboratorio-clinico-service
- imagenologia-service

El endpoint principal solicitado fue implementado:

```
GET /repositorio/paciente/{idPacienteRegional}
```

Tambien se agrego consulta por cedula:

```
GET /repositorio/cedula/{cedula}
```

3. Como lo hicimos

El repositorio usa comunicacion REST mediante RestTemplate. Las URLs de los microservicios se configuran por variables:

```
servicios.paciente-url  
servicios.consulta-url  
servicios.laboratorio-url  
servicios.imagenologia-url
```

En Docker Compose estas rutas apuntan a los nombres internos de los contenedores:

```
PACIENTE_SERVICE_URL: http://paciente-maestro-service:8081  
CONSULTA_SERVICE_URL: http://consulta-clinica-service:8082  
LABORATORIO_SERVICE_URL: http://laboratorio-clinico-service:8083  
IMAGENOLOGIA_SERVICE_URL: http://imagenologia-service:8084
```

Esto demuestra que la comunicacion se hace por APIs REST entre servicios y no por acceso directo entre bases de datos.

4. Diagrama actualizado de integracion

El diagrama de integracion REST esta disponible en:

documentacion/assets/diagrama_integracion_rest_solca.png

Flujo logico:

```
sequenceDiagram  
    participant Cliente as Cliente/Postman/Frontend  
    participant Repo as Repositorio Clinico Regional  
    participant Pac as Paciente Maestro  
    participant Con as Consulta Clinica  
    participant Lab as Laboratorio Clinico  
    participant Img as Imagenologia  
  
    Cliente->>Repo: GET /repositorio/paciente/{idPacienteRegional}  
    Repo->>Pac: GET /pacientes/{idPacienteRegional}  
    Repo->>Con: GET /consultas/paciente/{idPacienteRegional}  
    Repo->>Lab: GET /laboratorio/paciente/{idPacienteRegional}  
    Repo->>Img: GET /imagenes/paciente/{idPacienteRegional}  
    Repo-->>Cliente: JSON consolidado
```

5. Respuesta JSON consolidada

El endpoint devuelve el formato exigido:

```
{  
  "paciente": {},  
  "consultas": [],  
  "laboratorio": [],  
  "imagenes": [],  
  "errores": {}  
}
```

El campo errores se agrego para evidenciar manejo basico de fallos cuando algun microservicio no responde.

6. No acceso directo entre bases de datos

El cumplimiento se evidencia en la clase:

repositorio-clinico-regional-service/src/main/java/ec/edu/solca/repositorio/service/RepositorioIntegracionService.java

El repositorio no abre conexiones a paciente_maestro_db, consulta_clinica_db, laboratorio_clinico_db ni imagenologia_db. En lugar de eso llama endpoints REST:

- /pacientes/{idPacienteRegional}
- /consultas/paciente/{idPacienteRegional}
- /laboratorio/paciente/{idPacienteRegional}

- /imagenes/paciente/{idPacienteRegional}

Cada microservicio sigue siendo responsable de consultar su propia base.

7. Manejo basico de errores

Si un microservicio no responde, el repositorio captura la excepcion y devuelve una respuesta parcial. La informacion disponible de los demas servicios sigue apareciendo.

Ejemplo esperado si laboratorio esta apagado:

```
{
  "paciente": {},
  "consultas": [],
  "laboratorio": [],
  "imagenes": [],
  "errores": {
    "laboratorio-clinico": "No se pudo consultar el microservicio"
  }
}
```

Esto permite demostrar resiliencia basica y consolidacion parcial.

8. Infraestructura de datos

Tipo	Implementacion
Datos estructurados	Pacientes, consultas, laboratorio e imagenologia guardados en PostgreSQL.
Datos no estructurados	Archivos DICOM y metadatos de imagenologia en el microservicio de imagenologia.
DBaaS	PostgreSQL puede migrarse a un servicio administrado manteniendo las bases independientes.
Storage cloud	Propuesto para archivos DICOM, respaldos y objetos clinicos no estructurados.

9. Matriz preliminar de riesgos

Riesgo	Impacto	Mitigacion inicial
Microservicio no disponible	Respuesta incompleta	Manejo de errores por servicio y consolidacion parcial.
Duplicidad de pacientes	Informacion fragmentada	Uso de idPacienteRegional como identificador maestro.
Acceso indebido a datos clinicos	Exposicion de informacion sensible	Seguridad JWT reforzada en Avance 3.
Acoplamiento entre bases de datos	Dificultad de mantenimiento	Comunicacion REST y base independiente por microservicio.
Perdida de datos	Riesgo asistencial	PostgreSQL persistente y plan de respaldo en Avance 3.

10. Evidencias disponibles

Evidencia solicitada	Donde esta en el proyecto
Diagrama actualizado de integracion	documentacion/assets/diagrama_integracion_rest_solca.png.
Comunicacion REST entre servicios	RepositorioIntegracionService.java.
Respuesta JSON consolidada	Endpoint GET /repositorio/paciente/{idPacienteRegional}.
No acceso directo entre bases	Configuracion por URLs REST en docker-compose.yml.

Evidencia solicitada	Donde esta en el proyecto
Manejo de errores	Metodos obtenerObjeto y obtenerLista devuelven errores por servicio.
Pruebas Postman	postman/SOLCA-Repositorio-Clinico-Regional.postman_collection.json.

11. Cumplimiento de la rubrica

Requisito	Estado	Explicacion
Diagrama actualizado de integracion	Cumple	Existe diagrama REST en documentacion/assets.
Explicacion de APIs REST	Cumple	El repositorio consume endpoints HTTP de cada microservicio.
Infraestructura de datos	Cumple	Se documentan datos estructurados, DICOM, DBaaS y storage cloud.
Matriz preliminar de riesgos	Cumple	Incluida en este documento y en la documentacion general.
Consumo de Paciente Maestro	Cumple	GET /pacientes/{id} desde repositorio.
Consumo de Consulta Clinica	Cumple	GET /consultas/paciente/{id} desde repositorio.
Consumo de Laboratorio	Cumple	GET /laboratorio/paciente/{id} desde repositorio.
Consumo de Imagenologia	Cumple	GET /imagenes/paciente/{id} desde repositorio.
Endpoint obligatorio	Cumple	GET /repositorio/paciente/{idPacienteRegional} implementado.
Respuesta JSON consolidada	Cumple	Devuelve paciente, consultas, laboratorio, imagenes y errores.
No acceso directo entre bases	Cumple	Integracion por REST, no por consultas SQL cruzadas.
Manejo de errores si un servicio cae	Cumple	Devuelve informacion parcial y mensaje en errores.

12. Conclusion

El Avance 2 queda cubierto porque el repositorio regional integra los microservicios mediante REST, consolida la informacion clinica por paciente y maneja fallos parciales. La arquitectura respeta la independencia de bases de datos y prepara el sistema para seguridad, frontend y contenedores del Avance 3.

Anexos: diagramas

Como evidencia del diseño e implementación se incluyen los diagramas generados para el proyecto.

Diagrama actualizado de integración REST

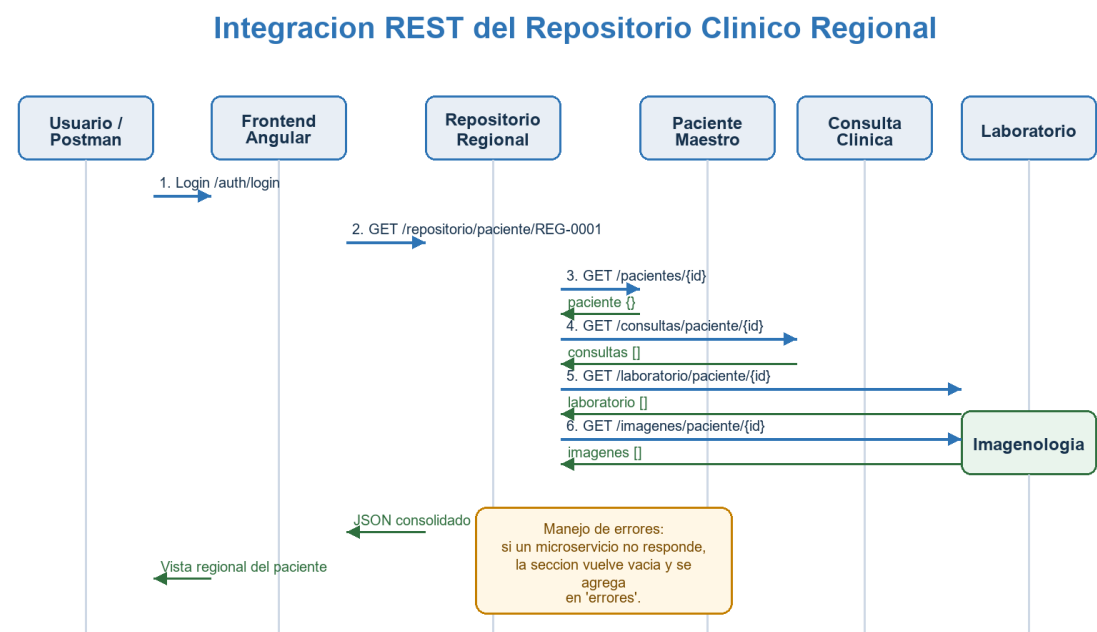
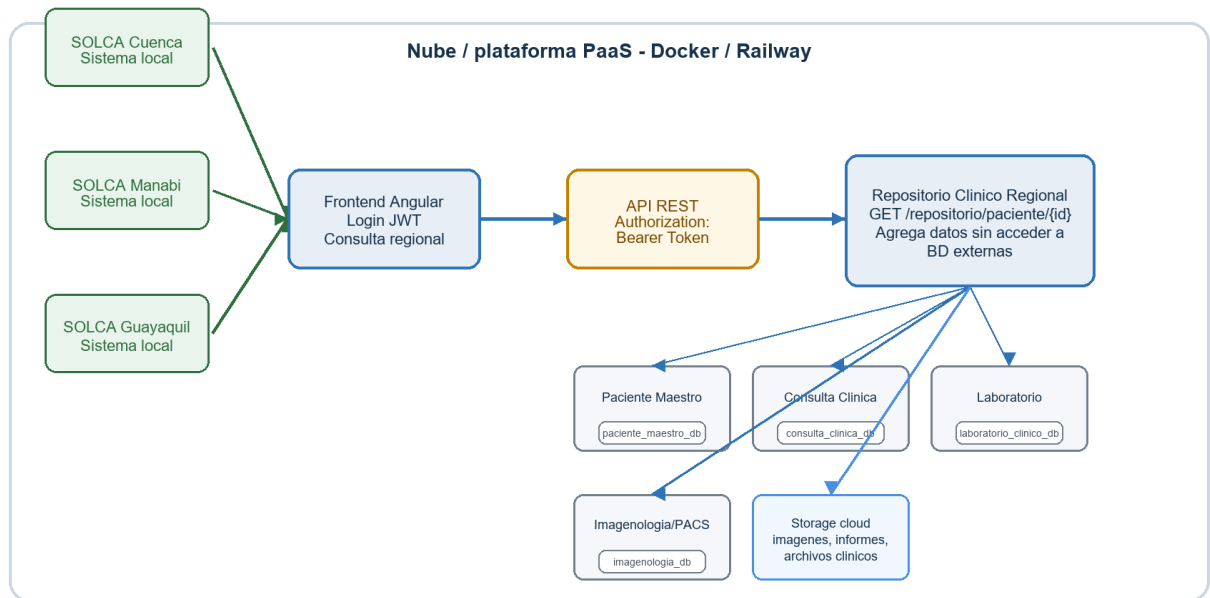


Diagrama actualizado de integración REST

Arquitectura cloud del repositorio regional

Arquitectura cloud preliminar - Repositorio Clinico Regional SOLCA



Arquitectura cloud del repositorio regional

 Universidad Católica de Cuenca	LISTA DE COTEJO PARA LA EVALUACIÓN DEL PROYECTO RETO
---	---

**UNIDAD ACADÉMICA DE INFORMÁTICA, CIENCIAS DE LA
COMPUTACIÓN, E INNOVACIÓN TECNOLÓGICA**
CARRERA DE SISTEMAS BIOMÉDICOS
Período Lectivo MARZO 2026 – AGOSTO 2026

Asignatura:	Infraestructura de Sistemas Clínicos en la Nube – Gestión de Proyectos Tecnológicos Sanitarios	Fecha:	20 de julio del 2026
Docente:	Ing. Fanny Chuchuca, MSc. Ing. Pablo Salamea, Msc.		
Ciclo:	7mo	Bloque Temático N°:	3
TEMA:	Diseño e Implementación de un Repositorio Clínico Único en la Nube para una Red Hospitalaria Oncológica Multi-Sede		
Nombre del estudiante:			

AVANCE 3 — Frontend, seguridad y contenedores
Repositorio Clínico Único en la Nube para una Red Hospitalaria Oncológica Multi-Sede

1. Objetivo del avance

El tercer avance tuvo como objetivo completar el sistema con interfaz grafica, seguridad por JWT/Spring Security, roles, auditoria basica y ejecucion en contenedores. En esta fase el repositorio ya no se prueba solo por Postman, sino tambien desde una aplicacion Angular desplegada en contenedor Nginx.

2. Que hicimos

Se implemento y conecto un frontend Angular con el Repositorio Clinico Regional. La aplicacion permite:

- Iniciar sesion con usuario y contrasena.
- Registrar nuevos usuarios clinicos.
- Buscar pacientes por cedula ecuatoriana o Master ID regional.
- Visualizar informacion clinica consolidada.
- Ver perfil del paciente.
- Ver historia clinica.
- Ver consultas clinicas.
- Ver laboratorio.

- Ver imagenología.
- Enviar y visualizar archivos DICOM cuando existen.
- Mostrar mensajes si algun microservicio no esta disponible.
- Consultar auditoria basica.

Tambien se agrego seguridad JWT con roles ADMIN, MEDICO y LABORATORIO, Dockerfile por servicio y despliegue por Docker Compose.

3. Como lo hicimos

Frontend

El frontend esta en el repositorio:

solca-repositorio-frontend

Archivos principales:

- src/app/app.html
- src/app/app.ts
- src/app/services/repositorio-clinico.ts
- Dockerfile
- nginx.conf

La aplicacion Angular consume el backend por rutas /api, que son redirigidas por Nginx al microservicio de repositorio regional.

Seguridad

La seguridad se implemento con:

- JWT.
- Spring Security.
- Filtros de autenticacion por token.
- Roles ADMIN, MEDICO, LABORATORIO.
- Control de acceso por endpoint con @PreAuthorize.
- Login en POST /auth/login.
- Registro de usuarios en POST /auth/register.

Cada microservicio contiene paquete security con:

- SecurityConfig.java
- JwtService.java
- JwtAuthenticationFilter.java
- JwtPrincipal.java

Auditoria

El repositorio registra auditoria basica de acciones relevantes. La auditoria contiene los campos minimos solicitados:

- usuario
- rol
- fecha
- hora
- direccion IP
- modulos
- paciente
- accion
- resultados

El frontend permite consultar auditoria desde el menu lateral.

Contenedores

Cada microservicio backend tiene su propio Dockerfile:

- paciente-maestro-service/Dockerfile
- consulta-clinica-service/Dockerfile
- laboratorio-clinico-service/Dockerfile
- imagenologia-service/Dockerfile
- historia-clinica-service/Dockerfile
- repositorio-clinico-regional-service/Dockerfile

El frontend tambien tiene Dockerfile:

- solca-repositorio-frontend/Dockerfile

La ejecucion integrada se realiza con:

- docker-compose.yml
- docker-compose.vps.yml

4. Diseno final de arquitectura

La arquitectura final queda compuesta por:

- Frontend Angular en Nginx.
- Repositorio Clinico Regional como API principal.
- Microservicios internos por dominio clinico.
- PostgreSQL con bases independientes.
- JWT compartido para validacion de seguridad.
- Red interna Docker para comunicacion entre servicios.

```
flowchart LR
  U[Usuario SOLCA] --> FE[Frontend Angular + Nginx]
  FE --> RR[Repositorio Clinico Regional]
  RR --> PM[Paciente Maestro]
  RR --> CC[Consulta Clinica]
  RR --> HC[Historia Clinica]
  RR --> LB[Laboratorio Clinico]
  RR --> IM[Imagenologia / PACS]
  PM --> DB1[(paciente_maestro_db)]
  CC --> DB2[(consulta_clinica_db)]
  HC --> DB3[(historia_clinica_db)]
  LB --> DB4[(laboratorio_clinico_db)]
  IM --> DB5[(imagenologia_db)]
  RR --> DB6[(repositorio_clinico_regional_db)]
```

5. Estrategia de seguridad

Elemento	Implementacion
Autenticacion	Login con usuario y contrasena, generacion de JWT.
Autorizacion	Roles ADMIN, MEDICO, LABORATORIO.
Proteccion de endpoints	@PreAuthorize en controladores.
Sesion	Stateless; el token viaja en Authorization: Bearer.
Secretos	JWT_SECRET configurado en Compose; en cloud debe migrarse a secret manager.
Validaciones	Cedula ecuatoriana, nombres con letras, campos obligatorios por formulario.

6. Estrategia de auditoria

La auditoria se implemento en el repositorio regional para registrar consultas y acciones importantes. Esto permite saber que usuario consulto, que rol tenia, desde que IP, que modulo uso, que paciente consulto y cual fue el resultado.

La vista de auditoria se encuentra en el frontend y consume:

```
GET /repositorio/auditoria
```

7. Matriz de riesgos actualizada

Riesgo	Probabilidad	Impacto	Mitigacion
Acceso no autorizado a informacion clinica	Media	Alto	JWT, roles y control por endpoint.
Caida de un microservicio	Media	Medio	Consolidacion parcial y mensajes de servicio no disponible.
Perdida de datos	Baja	Alto	Volumen persistente de PostgreSQL y plan de respaldo.
Exposicion de secretos	Media	Alto	Variables de entorno; recomendacion de secret manager en cloud.
Fallo de contenedores	Media	Medio	Dockerfile por servicio y Compose para reconstruccion.
Auditoria incompleta	Baja	Medio	Campos de auditoria ampliados: usuario, rol, fecha, hora, IP, modulo, paciente, accion y resultado.

8. Plan de respaldo y recuperacion

Respaldo recomendado:

```
docker exec solca-av3-postgres pg_dumpall -U solca_av3 > backup-solca-av3.sql
```

Restauracion recomendada:

```
cat backup-solca-av3.sql | docker exec -i solca-av3-postgres psql -U solca_av3 -d solca_admin
```

Frecuencia recomendada:

- Antes de cada entrega.
- Antes de cambios de esquema.
- Antes de cargar informacion masiva.
- Diario en ambiente productivo.

9. Diagrama con frontend, seguridad y cloud

El diagrama final se documenta en:

docs/avance-3-frontend-seguridad-contenedores.md

Tambien existen diagramas generados en:

- documentacion/assets/diagrama_arquitectura_cloud_solca.png
- documentacion/assets/diagrama_componentes_cloud_solca.png
- documentacion/assets/diagrama_integracion_rest_solca.png

10. Cumplimiento de interfaz grafica

Requisito	Estado	Explicacion
Buscar paciente por cedula o ID regional	Cumple	Dashboard permite buscar por cedula ecuatoriana o Master ID.
Visualizar informacion clinica consolidada	Cumple	El frontend consume el endpoint consolidado del repositorio.
Mostrar historias clinicas locales por sede	Cumple	Se agrego modulo de historia clinica con sede de registro/apertura.
Mostrar consultas	Cumple	Vista de consultas y registro de nuevas consultas.
Mostrar laboratorio	Cumple	Vista de laboratorio y registro segun rol.
Mostrar imagenologia	Cumple	Vista de imagenologia, envio DICOM y visor si hay archivo disponible.
Mostrar mensaje si un servicio no esta disponible	Cumple	Seccion "Servicios no disponibles" muestra errores por microservicio.

11. Cumplimiento de seguridad

Requisito	Estado	Explicacion
JWT o Spring Security	Cumple	JWT y Spring Security en backend.
Roles ADMIN, MEDICO, LABORATORIO	Cumple	Roles implementados en login, registro y control por endpoint.
Control de acceso por endpoint	Cumple	Uso de @PreAuthorize y filtros JWT.
Auditoria basica	Cumple	Registro y vista de auditoria con los campos solicitados.

12. Cumplimiento de contenedores

Requisito	Estado	Explicacion
Dockerfile para cada microservicio	Cumple	Cada microservicio backend tiene Dockerfile.
Evidencia de construccion de imagenes	Cumple tecnicamente	docker compose build construye las imagenes; se deben anexar capturas si el docente las exige.
Evidencia de ejecucion en contenedores	Cumple tecnicamente	docker compose up -d ejecuta los contenedores; se deben anexar capturas de docker ps.

13. Evidencias disponibles

Evidencia	Ruta o comando
Frontend Angular	Repositorio solca-repositorio-frontend.
Backend completo	Repositorio solca-repositorio-backend.
Docker Compose local	docker-compose.yml.
Docker Compose VPS	docker-compose.vps.yml.
Documentacion avance 3	docs/avance-3-frontend-seguridad-contenedores.md.
Auditoria	Endpoint GET /repositorio/auditoria.
Servicios disponibles	Endpoint GET /repositorio/servicios.
Contenedores	docker ps en el VPS o local.

Evidencia	Ruta o comando
Construccion de imagenes	docker compose -f docker-compose.vps.yml build.

14. Conclusion

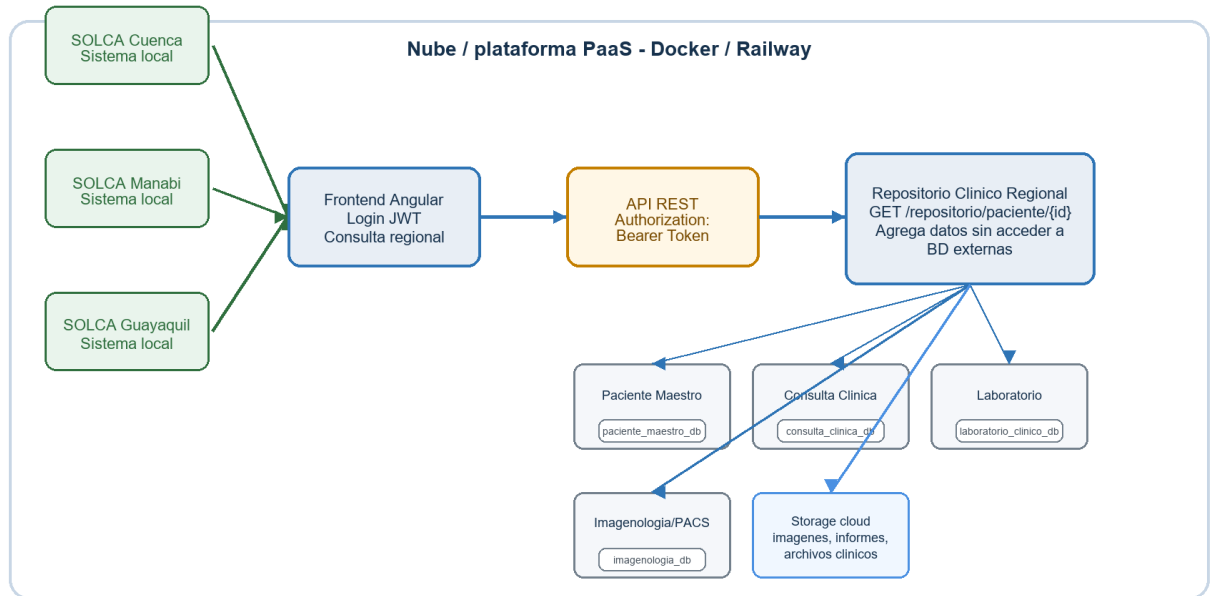
El Avance 3 queda cubierto porque el sistema ya cuenta con frontend funcional, autenticacion JWT, roles, control por endpoint, auditoria, validaciones, contenedores y despliegue en VPS. Las evidencias tecnicas existen en codigo y configuracion; para una entrega academica completa se recomienda anexar capturas de pantalla de login, busqueda, vistas clinicas, auditoria, construccion Docker y contenedores ejecutandose.

Anexos: diagramas

Como evidencia del diseño e implementación se incluyen los diagramas generados para el proyecto.

Arquitectura final con frontend, seguridad y cloud

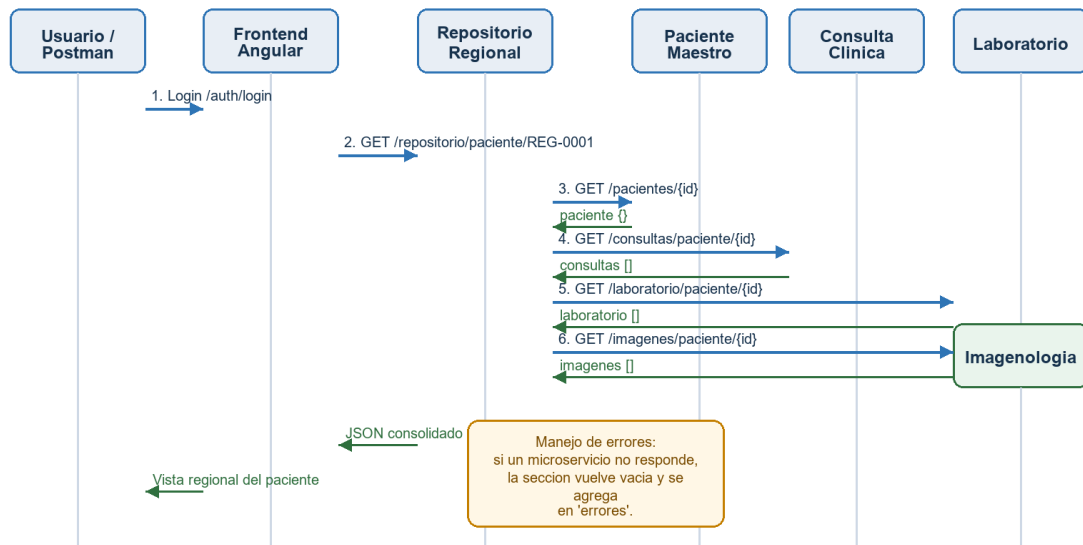
Arquitectura cloud preliminar - Repositorio Clinico Regional SOLCA



Arquitectura final con frontend, seguridad y cloud

Integración de microservicios mediante APIs REST

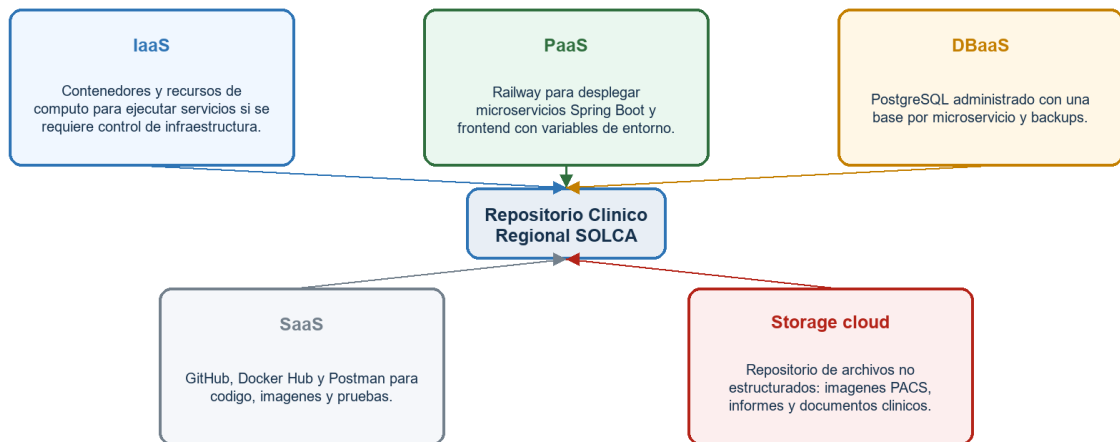
Integración REST del Repositorio Clínico Regional



Integración de microservicios mediante APIs REST

Componentes cloud y contenedores

Componentes cloud identificados para el proyecto



Componentes cloud y contenedores